

PREDICTIVE MODEL PLAN

1) Model Logic (Generated with GenAI)

I used ChatGPT (GenAI) to generate the structure and logic of my predictive model.

Model Type:

Random Forest Classifier — a tree-based ensemble model. Tree-based models naturally handle: Categorical + numerical data, Missing values (to an extent), Outliers, Non-monotonic relationships (like Missed_Payments)

Key Input Features and Contribution to Predictions:

Feature	Contribution (Based on EDA Insights)
Missed_Payments	Complex non-linear impact — specific thresholds may increase risk. Key signal of delinquency.
Employment_Status	Unemployed customers had higher delinquency — an important categorical risk indicator.
Credit_Card_Type	'Business' card holders had much higher delinquency — a strong segment risk indicator.
Debt_to_Income_Ratio	Higher ratio generally increases risk — contributes to credit stress.
Credit_Utilization	High utilization correlates weakly with risk — still a useful signal of overextension.
Credit_Score	Weak positive correlation — helps identify creditworthiness.
Income	Helps moderate other risk signals — very high/very low income could both impact risk.
Loan_Balance	Combined with Income and Debt Ratio, contributes to financial stress profile.
Account_Tenure	Surprisingly weak negative correlation — may still act as a slight stability indicator.

Location	Regional differences found (e.g. Los Angeles higher risk) — could help capture unobserved economic factors.
Month_1 to Month_6	Payment behavior trend — extremely important for modeling delinquency. Temporal signal of increasing or decreasing risk.

General Workflow of the Model:

1 Data Ingestion

- Load the dataset (as cleaned from EDA — with imputed values, consistent categorical encoding).

2 Feature Engineering

- Create additional useful features, such as:
 - Total count of 'Late' and 'Missed' in Month_1 to Month_6.
 - Trend of payment history (getting better or worse).
 - Composite risk score combining Credit Utilization, Debt Ratio, Credit Score.
- Encode categorical variables (Employment_Status, Credit_Card_Type, Location) using one-hot encoding or label encoding.

3 Model Training

- Train a Random Forest / XGBoost model to predict Delinquent_Account (binary classification: 0/1).
- Use cross-validation to tune model hyperparameters and prevent overfitting.

4 Model Inference / Prediction

- For each new customer record:
 - Process the input features.
 - Feed into trained models.
 - Model outputs probability of delinquency ($P[\text{Delinquent_Account}=1]$).
 - Classify as delinquent or not based on threshold (typically 0.5, but can be adjusted).

5 Model Interpretability

- Use feature importance scores from Random Forest/XGBoost to understand what is driving predictions.
- Optionally use SHAP values to explain individual predictions.

Predictive Modeling Pipeline for Credit Delinquency Forecasting:

Step 1: Feature Selection

- Based on EDA, select the most informative features:
 - Income
 - Credit_Utilization
 - Missed_Payments
 - Employment_Status
 - Debt_to_Income_Ratio
 - Credit_Score
 - Loan_Balance
 - Account_Tenure
 - Credit_Card_Type
 - Location
 - Month_1 to Month_6 (create summary features from these)

Step 2: Data Preprocessing

- Handle missing values:
 - Income: Use synthetic or imputed values.
 - Credit_Score & Loan_Balance: Impute with median.
- Convert categorical variables:
 - Employment_Status, Credit_Card_Type, Location → one-hot encoding.
- Process Month_1 to Month_6: Create features like:
 - Total_Late_Payments_6M
 - Total_Missed_Payments_6M
 - Payment_Behavior_Trend

Step 3: Model Training

- Split data: 70% train, 30% test (or use cross-validation).

- Train selected model (see next section for options).
- Tune hyperparameters for optimal performance.

Step 4: Model Evaluation

- Evaluate on holdout test set using:
 - Accuracy
 - Precision
 - Recall
 - F1-Score
 - AUC-ROC (Area Under ROC Curve) → to measure how well the model separates classes.
- Analyze feature importance.

Two Modeling Options (Simple + Complex), and Recommendation

Option	Model Type	Pros	Cons	Recommended
Simple	Logistic Regression	Easy to interpret, fast, baseline model	Can't capture non-linear relationships well	Useful as baseline
Complex	Random Forest / XGBoost (Tree-Based)	Captures complex patterns, handles feature interactions, robust to outliers	More complex, less interpretable	Recommended

How the Delinquency Risk Model Works — End-to-End Transformation

Stage	Description
Data Ingestion	Load raw customer data (with cleaned/imputed fields).
Feature Transformation	Create engineered features from payment history, encode categoricals, normalize if needed.
Model Input	Transformed numerical and encoded categorical feature matrix (X).

Model Processing	Feed data through trained Random Forest/XGBoost model. The model computes learned decision trees and outputs a probability score for each record.
Model Output	For each customer: probability of delinquency → classify as 1 (high risk) or 0 (low risk) based on chosen threshold (e.g., 0.5).
Result Delivery	Output used in credit risk dashboards or decision engines to flag risky customers for further review.

What the model is designed to do:

The model is designed to predict whether a customer is likely to become delinquent (binary target: 0 = not delinquent, 1 = delinquent) based on financial and behavioral attributes. The goal is to support Geldium's credit risk management by identifying high-risk customers for proactive interventions.

Import libraries

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import OneHotEncoder
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, roc_auc_score
```

Load the dataset

```
df = pd.read_csv("Delinquency_prediction_dataset.csv")
```

Impute missing values (example)

```
df['Income'].fillna(df['Income'].mean(), inplace=True)
df['Credit_Score'].fillna(df['Credit_Score'].median(), inplace=True)
df['Loan_Balance'].fillna(df['Loan_Balance'].median(), inplace=True)
```

Feature engineering: Month_1 to Month_6

```
month_cols = ['Month_1', 'Month_2', 'Month_3', 'Month_4', 'Month_5', 'Month_6']
df['Total_Late_Payments_6M'] = df[month_cols].apply(lambda row: (row == 1).sum(), axis=1)
df['Total_Missed_Payments_6M'] = df[month_cols].apply(lambda row: (row == 2).sum(), axis=1)
```

One-hot encode categorical columns

```

categorical_cols = ['Employment_Status', 'Credit_Card_Type', 'Location']
df = pd.get_dummies(df, columns=categorical_cols, drop_first=True)

# Select features and target
features = [
    'Income', 'Credit_Utilization', 'Missed_Payments',
    'Debt_to_Income_Ratio', 'Credit_Score', 'Loan_Balance',
    'Account_Tenure', 'Total_Late_Payments_6M', 'Total_Missed_Payments_6M'
] + [col for col in df.columns if any(prefix in col for prefix in categorical_cols)]

X = df[features]
y = df['Delinquent_Account']

# Split data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
random_state=42)

# Train Random Forest model
rf_model = RandomForestClassifier(n_estimators=100, random_state=42)
rf_model.fit(X_train, y_train)

# Predict and evaluate
y_pred = rf_model.predict(X_test)
y_prob = rf_model.predict_proba(X_test)[:, 1]

# Evaluation metrics
print(classification_report(y_test, y_pred))
print(f"AUC-ROC: {roc_auc_score(y_test, y_prob):.2f}")

```

Summary of Chosen Model:

We recommend using a Random Forest Classifier for credit delinquency prediction, as it effectively captures complex, non-linear relationships found in the dataset.

This model handles mixed data types, feature interactions, and outliers well, providing a robust and interpretable framework for predicting customer risk.

An initial baseline logistic regression can be used for comparison but is expected to perform worse due to weak linear correlations.

Top 5 Input Features:

1. Missed_Payments
2. Employment_Status
3. Credit_Card_Type
4. Debt_to_Income_Ratio
5. Credit_Utilization

2) Justification for Model Choice

Comparison of Logistic Regression vs. Decision Trees for Financial Risk Prediction:

Aspect	Logistic Regression	Decision Trees (Random Forest / XGBoost)
Interpretability	Very high (easy to explain coefficients)	Good (feature importance, SHAP values help explain decisions)
Ability to Capture Complex Patterns	Poor — only linear relationships	Excellent — captures non-linear patterns & feature interactions
Handling Outliers	Sensitive to outliers	Robust to outliers
Handling Missing Data	Requires imputation	More tolerant of missing values (depends on implementation)
Training Speed	Very fast	Slower but manageable for small-medium datasets
Prediction Speed	Very fast	Fast (slightly slower than logistic regression, but acceptable)
Scalability	Excellent	Very good (can scale well with optimized implementations like XGBoost)
Ease of Deployment	Very easy	Slightly more complex but still practical (especially with scikit-learn or XGBoost)

Regulatory Transparency	High	Acceptable with feature importance reporting
-------------------------	------	--

I selected Random Forest classifier as the primary model for this task, as it offers the best trade-off between accuracy, transparency, and operational efficiency:

- **Accuracy:** Captures complex, non-linear patterns observed in the EDA (e.g., non-monotonic relationships in Missed_Payments, segment-specific risks in Credit_Card_Type, and location effects).
- **Transparency:** While less interpretable than logistic regression, Random Forest provides clear feature importance and SHAP values — sufficient to meet regulatory and business transparency needs.
- **Ease of Use:** Widely implemented and well-supported in Python’s scikit-learn library. Deployment and monitoring are operationally feasible.
- **Relevance for Financial Prediction:** Tree-based models are an industry-standard in credit risk modeling due to their ability to handle mixed data types, outliers, and missing values.
- **Fit for Geldium:** The model aligns with Geldium’s business goal of proactively identifying delinquency risk while maintaining fairness and transparency. It supports compliance with regulations and allows internal stakeholders to understand why customers are flagged as high risk.

3) Evaluation Strategy

Evaluation Strategy for Financial Risk Prediction Model — Fairness, Bias, and Accuracy

Category	Metric	Why It Matters	How to Interpret
Accuracy	Accuracy	Basic correctness — % of correct predictions.	Useful when classes are balanced; in credit risk, should not be the sole metric.
Model Performance	F1 Score	Balance of Precision & Recall — key for credit risk where false negatives and false positives both matter.	High F1 means balanced model; if $F1 < 0.65 \rightarrow$ revisit model tuning.

	AUC-ROC	Measures model's ability to distinguish delinquent vs. non-delinquent accounts.	AUC > 0.75 → strong separation; AUC < 0.7 → model may not be learning enough.
Precision	Precision	% of predicted "high risk" customers who are actually delinquent.	High precision avoids harming "safe" customers.
Recall	Recall	% of actual delinquent customers caught by model.	High recall avoids missing risky customers. Critical in credit risk!
Fairness Bias /	Group-wise F1, Precision, Recall	Compare performance across customer groups (Employment_Status, Location, Age groups).	Large gaps (≥10–15%) indicate potential bias. Address via mitigation.
	Demographic Parity	Are "high risk" predictions distributed fairly across groups?	Should not disproportionately classify a group as "high risk."
	Equal Opportunity	Are delinquent customers equally likely to be flagged across groups?	Helps ensure fairness in Recall per group.

Example Bias Mitigation Techniques in Credit Risk Modeling

Technique	When to Use	Example Application
Re-weighting	Data shows imbalance across groups (protected groups under/overrepresented).	Adjust sample weights during training so that the model treats groups fairly.

Pre-processing	Features may encode bias (e.g., Location acting as a proxy for protected attribute).	Remove problematic features or transform features to remove group correlation.
In-processing	Want to incorporate fairness constraints directly into training.	Use algorithms that optimize both accuracy and fairness (ex: fair-aware Random Forest or Fairlearn library).
Post-processing	Model already trained, want to adjust thresholds to improve fairness.	Use tools like Calibrated Equalized Odds to adjust decision thresholds for fairness.
Feature Auditing	Before modeling, ensure features do not encode or proxy protected attributes.	with demographics →may

Evaluation Strategy Summary

To ensure the model is both accurate and fair, I will adopt a two-level evaluation approach:

Level 1 — Performance Metrics:

- Use F1 Score and AUC-ROC as primary indicators of model performance.
- Track Precision and Recall to ensure good balance in catching risky customers while minimizing false positives.

Level 2 — Fairness Checks:

- Conduct group-wise analysis:
 - Compare F1, Precision, and Recall across key customer groups: Employment_Status, Location, Age groups.
- Apply Demographic Parity and Equal Opportunity checks to ensure the model does not unfairly penalize or ignore specific segments.

- If fairness issues are detected, apply bias mitigation techniques (such as re-weighting or post-processing) to improve fairness without sacrificing model accuracy.

Interpretation Guidelines:

- Target F1 Score ≥ 0.65 and AUC ≥ 0.75 .
- Target $\leq 10\text{--}15\%$ performance gap between customer groups.
- Ensure business stakeholders and compliance teams review fairness results before deployment.

Ethical Considerations:

- The model will avoid using features that directly encode sensitive demographic attributes (e.g., gender or ethnicity), or indirect proxies if they are found to introduce bias.
- Bias mitigation techniques such as **re-weighting**, **pre-processing feature auditing**, and **post-processing fairness adjustments** will be considered if significant group-wise performance gaps ($>10\text{--}15\%$) are detected.
- All fairness results will be reviewed with Geldium's compliance and business teams prior to deployment to ensure ethical, transparent, and compliant use of AI in credit risk prediction.